

Technical Reference: Data Structures Fundamentals

Educational Series

December 29, 2025

1 Introduction

A data structure is a specialized format for organizing, processing, retrieving, and storing data. Choosing the right data structure is crucial for writing efficient algorithms and software.

2 Classification of Data Structures

2.1 Linear Data Structures

Elements are arranged in a sequence or a linear list.

- **Arrays:** Fixed-size, contiguous memory locations.
- **Linked Lists:** Elements (nodes) containing data and a pointer to the next node.
- **Stacks:** Follows the LIFO (Last In, First Out) principle.
- **Queues:** Follows the FIFO (First In, First Out) principle.

2.2 Non-Linear Data Structures

Elements are not in a sequence but are connected in a hierarchical or network-like manner.

- **Trees:** Hierarchical structure with a root and child nodes (e.g., Binary Search Tree).
- **Graphs:** A set of vertices (nodes) and edges connecting them.

3 Linear Structures in Detail

3.1 Linked List

- **Singly Linked List:** Each node points to the next.
- **Doubly Linked List:** Each node points to both the previous and next nodes.

3.2 Stack and Queue

4 Non-Linear Structures in Detail

4.1 Binary Search Tree (BST)

In a BST, for every node:

Feature	Stack	Queue
Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Operations	Push, Pop, Peek	Enqueue, Dequeue
Usage	Recursion, Undo/Redo	Scheduling, Buffering

- The value of the **left** child is less than the parent.
- The value of the **right** child is greater than the parent.

4.2 Hashing

Hashing uses a hash function to map data of arbitrary size to fixed-size values (hash codes), enabling constant time ($O(1)$) average-case lookup.

5 Complexity Table (Average Case)

Data Structure	Access	Search	Insertion
Array	$O(1)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$
BST (Balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash Table	N/A	$O(1)$	$O(1)$

6 Choosing a Data Structure

- Use **Arrays** for fast access to elements via indices.
- Use **Linked Lists** if you need frequent insertion and deletion at the beginning.
- Use **Hash Tables** for high-performance searching.
- Use **Trees** for hierarchical data or sorted data retrieval.